

K-디지털 트레이닝

Vue3 Stomp

[KB] IT's Your Life

1 Vue3 Stomp

✓ socket.io vs WebSocket

○ socket.io

- 양방향 통신을 하기 위해 웹소켓 기술을 활용하는 라이브러리
- 방 개념으로 일부 클라이언트에게만 전송하는 브로드캐스팅이 가능함

○ WebSocket

- HTML5 웹 표준 기술
- 양방향 소통을 위한 프로토콜
- 매우 빠르게 작동하며 통신할 때 아주 적은 데이터를 이용함

1 Vue3 Stomp

✓ STOMP

- 웹 소켓은 양방향 통신이 가능하지만, 미지원 웹 브라우저가 있고, 웹 브라우저 이외의 클라이언트 지원에 대한 이슈 있음
- 따라서 Spring을 이용한 백엔드와의 통신에는 STOMP를 이용
- socket.js(sockjs)
 - 웹소켓과 유사한 객체를 제공하는 브라우저 js 라이브러리 .
 - 웹소켓이 지원하지 않는 브라우저까지 커버가능
- Stomp
 - 특정 방 토픽 을 구독을 하고 누군가 해당 방에 메시지를 send 하면 구독하고 있는 사용자는 해당 메시지를 수신 받음
- 주의 사항
 - 중복으로 구독하는 것을 방지하기 위해 구독코드를 최상단 요소에 위치하거나 unsubscribe 와 disconnect 를 이용해야 함
- 새로고침은 기존 구독이 끊기지만 페이지 이동은 기존 구독을 유지하면서 중복으로 구독될 수 있음

1 Vue3 Stomp

✓ @stomp/stompjs

- npm i @stomp/stompjs

- 사용법

- 초기화

- ```
import { Client } from '@stomp/stompjs';
let client = new Client(옵션객체);
```

- 연결

- ```
client.activate();
```

- 연결 끊기

- ```
client.deactivate();
```

# 1 Vue3 Stomp

## ✓ @stomp/stompjs

### ○ 옵션 객체

```
{
 brokerURL: 'ws://서버ip:포트번호/엔드포인트',
 onConnect: () => {},
 debug: (str) => {},
 onDisconnect: (frame) => {},
 onStompError: (frame) => {},
 onWebSocketClose: (_) => {},
 onWebSocketError: (_) => {},
}
```

# 1 Vue3 Stomp

## ✓ 메시지 전송

- `client.publish({ destination, body });`
  - destination: 토픽 문자열
  - body: 메시지, 주로 JSON 문자열

## ✓ 구독 신청 및 구독 처리

- `client.subscribe(topic, (frame) => { })`
  - topic: 구독할 토픽
  - `frame => {}` : 구독 수신 처리 함수
    - `frame : { destination, body, ... }`
  - `frame.body`를 `JSON.parse()`로 역직렬화해서 처리

# 1 Vue3 Stomp

## App.vue

```
<script setup>
import { ref } from 'vue';
import { Client } from '@stomp/stompjs';

const connected = ref(false);
const name = ref('');
const greetings = ref([]);
```

# 1 Vue3 Stomp

## App.vue

```
let client = new Client({
 brokerURL: 'ws://localhost:8080/gs-guide-websocket',
 debug: (str) => console.log(str),
 onConnect: () => {
 connected.value = true;
 client.subscribe('/topic/greetings', (frame) => {
 let greeting = JSON.parse(frame.body).content;
 greetings.value.push(greeting);
 });
 },
 onDisconnect: (frame) => {
 console.log('disconnected');
 connected.value = false;
 },
 onStompError: (frame) => {},
 onWebSocketClose: (_) => {},
 onWebSocketError: (_) => {},
});
```



# 1 Vue3 Stomp

## App.vue

```
const onConnect = () => {
 client.activate();
};

const onDisconnect = () => {
 client.deactivate();
};

const sendName = () => {
 const destination = '/app/hello';
 const body = JSON.stringify({ name: name.value });
 client.publish({ destination, body });
};
</script>
```

# 1 Vue3 Stomp

## App.vue

```
<template>
 <div>연결상태: {{ connected ? '연결됨' : '끊김' }}</div>

 <button v-if="!connected" @click="onConnect">연결</button>
 <button v-else @click="onDisconnect">끊기</button>
 <hr />

 <input type="text" v-model="name" /> <button @click="sendName" :disabled="!connected">전송</button>
 <hr />

 <h3>Greetings</h3>
 <div>
 <div v-for="greet in greetings" :key="greet">
 {{ greet }}
 </div>
 </div>
</template>
```